

PianoDSL: A Domain-Specific Language for Musical Visualization

Esteban Alejandro Villalba Delgadillo, Daniel Felipe Barrera Suárez

Department of Computer Engineering, Universidad Distrital Francisco José de Caldas, Bogotá D.C.

{evillalbad, dbarreras}@udistrital.edu.co

Abstract

PianoDSL is a domain-specific language (DSL) designed for music composition and visualization. It offers an easy approach for people unfamiliar with traditional music creation, allowing them to bridge the gap between technical input and practical output.

Introduction

Domain-specific languages (DSLs) are valuable educational tools that simplify complex concepts through specialized syntax. While languages like Sonic Pi and Tidal-Cycles allow for real-time music manipulation through code, they often present a steep learning curve.

PianoDSL offers a more accessible pedagogical approach by using minimalist text commands to generate graphical outputs. The core idea is to translate a simple instruction like:

```
play C4 for 1.0 at 0.0
```

into a standard piano roll visualization, a common format in digital music production.

Language Definition

The structure of PianoDSL is defined by a formal context-free grammar. This grammar dictates the valid syntax for all instructions in the language.

Tokens

The lexical analyzer recognizes the following tokens:

- **KEYWORD:** ‘play’, ‘for’, ‘at’
- **NOTE:** e.g., ‘C4’, ‘F5’, ‘Db3’
- **NUMBER:** e.g., ‘1.0’, ‘0.5’

Context-Free Grammar (BNF)

The syntax is validated against the following rules:

```
1 # GRAMMAR DEFINITION FOR PIANO DSL:
2
3 # <S>          -> <INSTRUCTION> <S> | <INSTRUCTION>
4 # <INSTRUCTION> -> "play" <NOTE> "for" <NUMBER> "at" <NUMBER>
5 # <NOTE>        -> <LETTER> [<ACCIDENTAL>] <OCTAVE>
6 # <LETTER>      -> "A" | "B" | "C" | "D" | "E" | "F" | "G"
7 # <ACCIDENTAL> -> "#" | "b"
8 # <OCTAVE>      -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8"
9 # <NUMBER>       -> <DIGITS> ["." <DIGITS>]
10 # <DIGITS>      -> <DIGIT> <DIGITS> | <DIGIT>
11 # <DIGIT>       -> "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Figure 1: Context-free grammar for PianoDSL in Backus-Naur Form.

Compiled program

The compiler features a text editor interface where users can input musical instructions. For example:

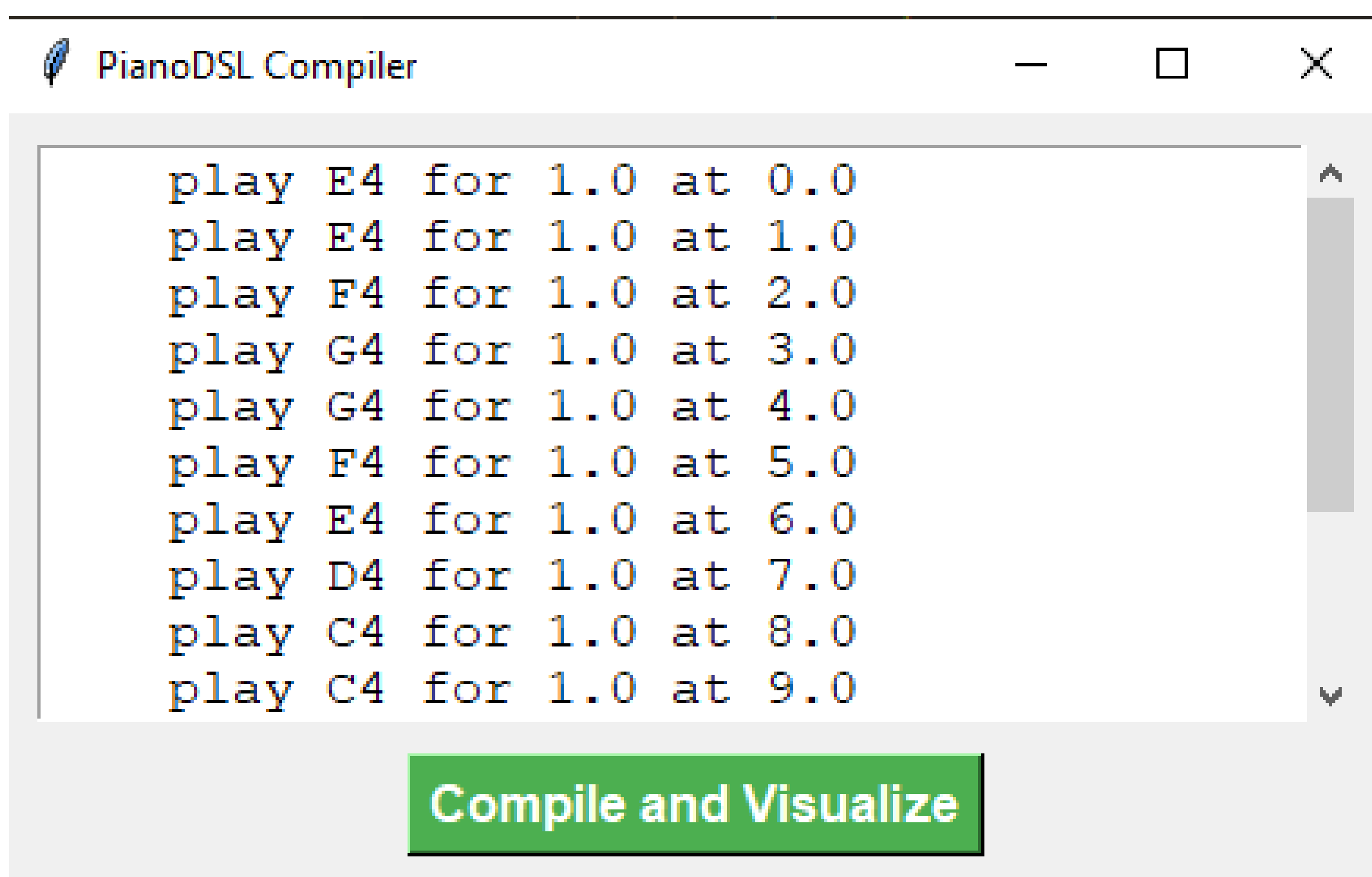


Figure 2: PianoDSL Compiler with examples.

It produces the piano roll visualization shown below, where each note is represented as a horizontal bar whose position and length correspond to its start time and duration.

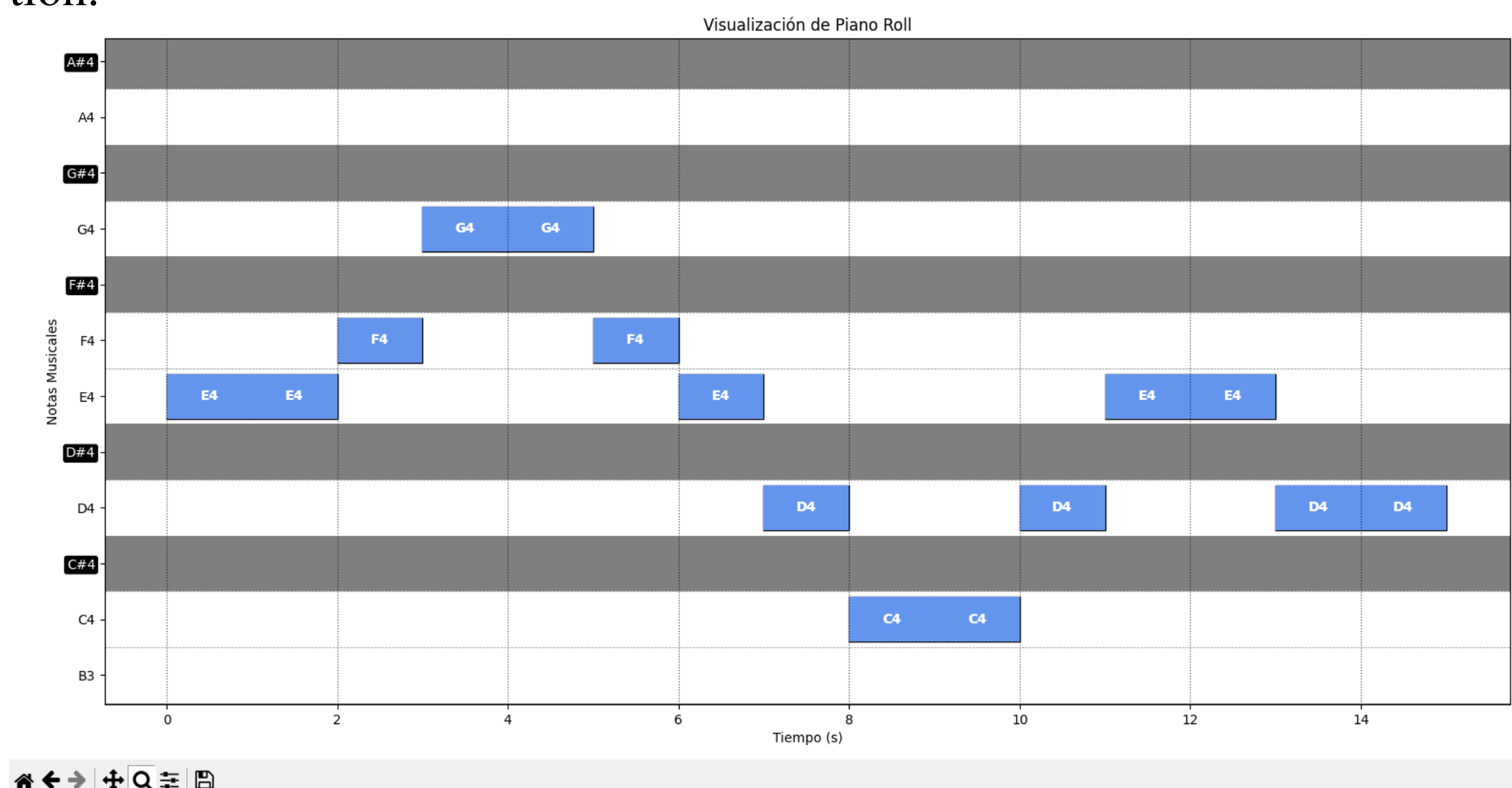


Figure 3: Sample piano roll generated by the PianoDSL compiler.

Benefits

- Easy-to-read musical commands
- Educational approach for beginners
- Modular compiler structure

Common errors and limitations

PianoDSL relies on strict grammar rules, so it would be common for users to get

errors like:

- **Unexpected character:** happens when a character doesn’t follow the grammar rules like adding caps to the Keywords or writing wrong a word.
- **Syntax Error:** happens when the user doesn’t add a word or number next to a Keyword.

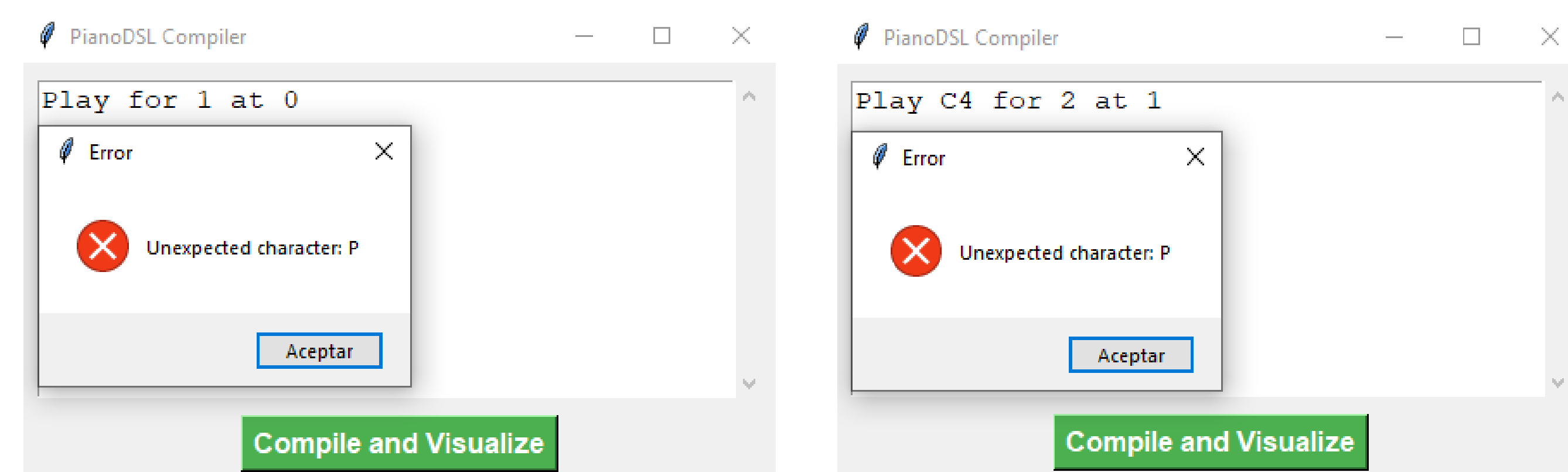


Figure 4: The program won’t compile until the error is solved.

Limitations

The current version of PianoDSL has several limitations:

- **Monophonic Only:** Does not support chords or concurrent notes (polyphony).
- **No Audio Output:** The system is purely visual in its current implementation.
- **Fixed Musical Parameters:** Lacks support for tempo, volume, or instrument changes.

Conclusion

PianoDSL successfully integrates lexical analysis, parsing, and data visualization to create a functional and educational tool. The project demonstrates that compiler design principles can be applied to creative fields like music, offering an accessible way to explore the intersection of programming and artistic expression

References

References

- [1] Sonic Pi. *The Live Coding Music Synth for Everyone*. <https://sonic-pi.net>
- [2] FoxDot. *Live-coding with Python and SuperCollider*. <https://foxdot.org>